

3. Create an Dockerised Python or Node.js application

Step-1: Create a Dockerfile using **nano** in the pwd

—> **Dockerfile**

```
FROM python:latest
WORKDIR /app
COPY req.txt .
RUN pip install --no-cache-dir -r req.txt
COPY . .
EXPOSE 5000
CMD ["python","app.py"]
```

Step-2: Create a python file **app.py**

→**app.py**

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello, Dockerrr...!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step-3: Create a requirements file **req.txt**

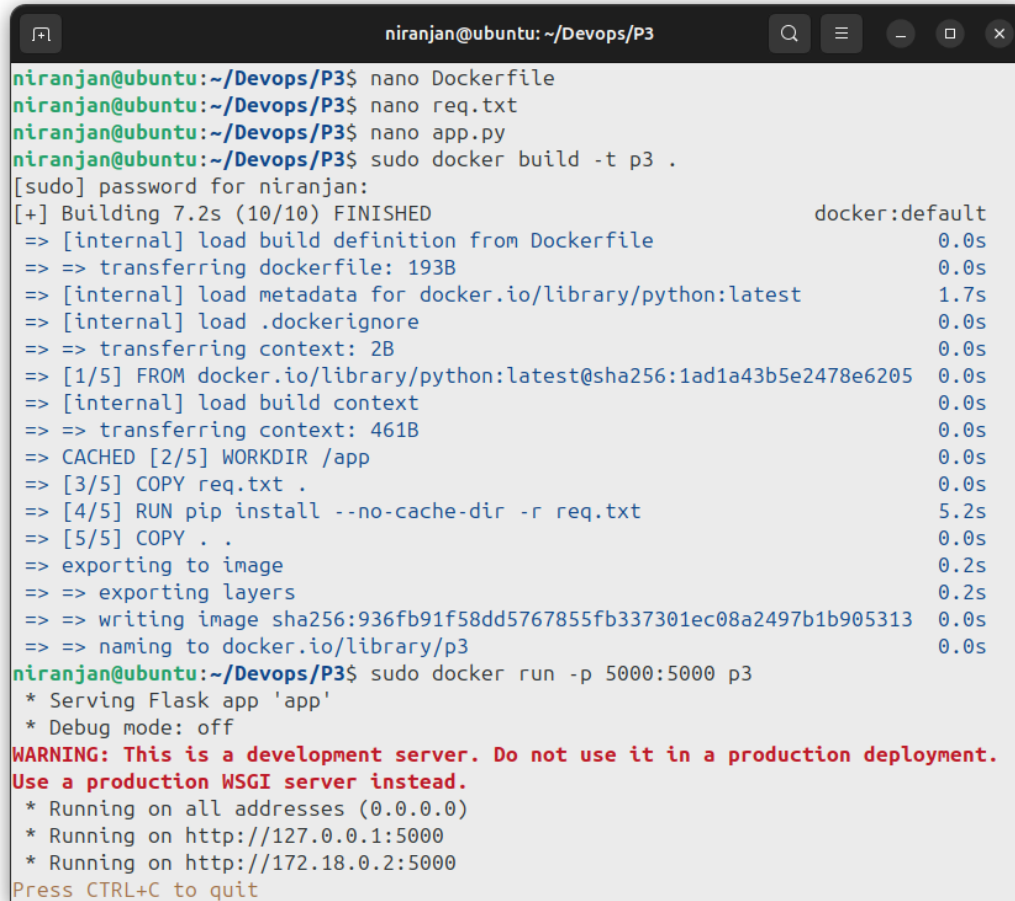
flask

Step-4: Build the docker image using the below command

```
docker build -t p1 .
```

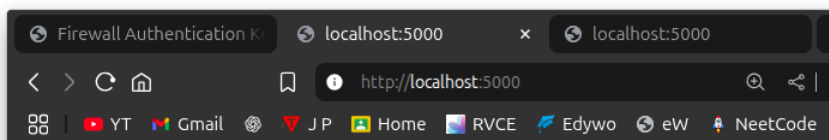
Step-5: Run the Docker Run command to create a container specifying the port number and name for the container

```
sudo docker run -p 5000:5000 p3
```

A terminal window titled 'niranjan@ubuntu: ~/Devops/P3' showing the process of building and running a Docker container. The user runs 'nano Dockerfile', 'nano req.txt', and 'nano app.py' before executing 'sudo docker build -t p3 .'. The build process is shown with progress bars and timestamps, including steps like 'load build definition', 'transferring dockerfile', 'load metadata', 'load .dockerignore', 'transferring context', 'FROM docker.io/library/python:latest', 'load build context', 'transferring context', 'WORKDIR /app', 'COPY req.txt', 'RUN pip install --no-cache-dir -r req.txt', and 'COPY .'. After the build is complete, the user runs 'sudo docker run -p 5000:5000 p3'. The output shows 'Serving Flask app 'app'', 'Debug mode: off', and a warning: 'WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.' It also lists the running addresses: 'Running on all addresses (0.0.0.0)', 'Running on http://127.0.0.1:5000', and 'Running on http://172.18.0.2:5000'. The prompt 'Press CTRL+C to quit' is at the bottom.

```
niranjan@ubuntu:~/Devops/P3$ nano Dockerfile
niranjan@ubuntu:~/Devops/P3$ nano req.txt
niranjan@ubuntu:~/Devops/P3$ nano app.py
niranjan@ubuntu:~/Devops/P3$ sudo docker build -t p3 .
[sudo] password for niranjan:
[+] Building 7.2s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 193B                                0.0s
=> [internal] load metadata for docker.io/library/python:latest    1.7s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:latest@sha256:1ad1a43b5e2478e6205 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 461B                                     0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> [3/5] COPY req.txt .                                           0.0s
=> [4/5] RUN pip install --no-cache-dir -r req.txt                5.2s
=> [5/5] COPY . .                                                 0.0s
=> exporting to image                                              0.2s
=> => exporting layers                                              0.2s
=> => writing image sha256:936fb91f58dd5767855fb337301ec08a2497b1b905313 0.0s
=> => naming to docker.io/library/p3                               0.0s
niranjan@ubuntu:~/Devops/P3$ sudo docker run -p 5000:5000 p3
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.2:5000
Press CTRL+C to quit
```

Step-6: Test the container by verifying whether its showing some results on <http://localhost:5000> stating **“hello Dockerr!”**



Hello, Dockerrr...!

You can also check the status of the container by using below command

`docker ps` // displays current running containers

`Docker ps -a` // displays all present containers

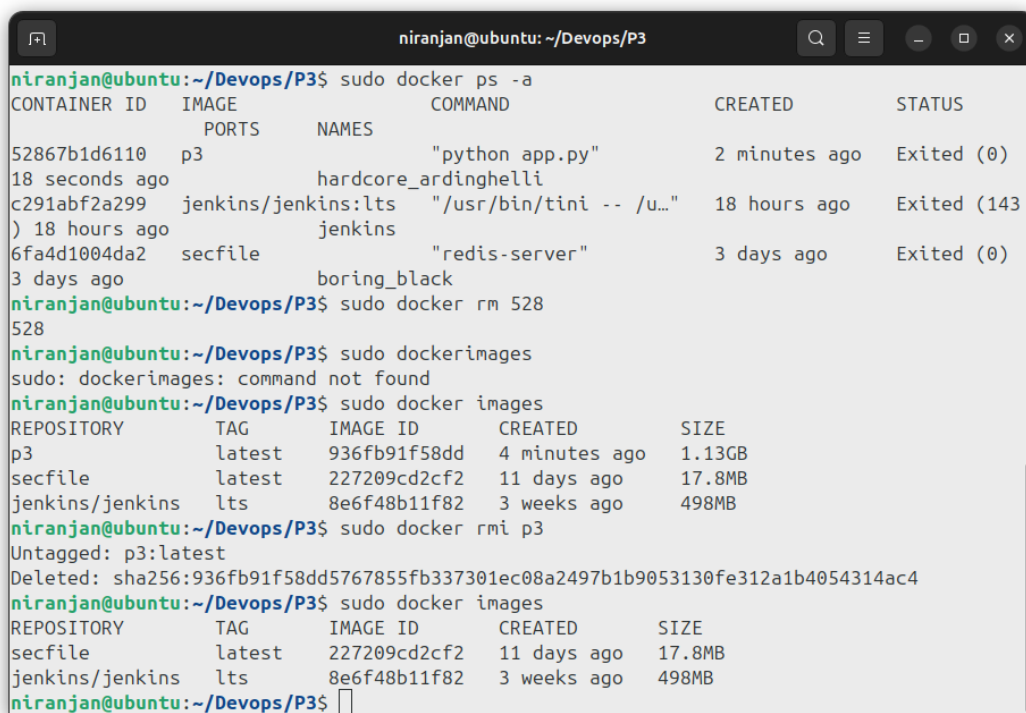
Step-7: Stop the container, remove the container using container number

`sudo docker container ls -a` or `docker ps -a`

`sudo docker container stop <container-id>`

`sudo docker container rm <container-id>`

`sudo docker image rm <image-id>` or `docker rmi <image-id>`



```
niranjan@ubuntu: ~/Devops/P3
niranjan@ubuntu:~/Devops/P3$ sudo docker ps -a
CONTAINER ID   IMAGE      PORTS    NAMES    COMMAND    CREATED    STATUS
52867b1d6110   p3        p3       hardcore_ardinghelli    "python app.py"    2 minutes ago    Exited (0)
18 seconds ago
c291abf2a299   jenkins/jenkins:lts    "/usr/bin/tini -- /u..."    18 hours ago    Exited (143)
) 18 hours ago    jenkins
6fa4d1004da2   secfile    boring_black    "redis-server"    3 days ago    Exited (0)
3 days ago
niranjan@ubuntu:~/Devops/P3$ sudo docker rm 528
528
niranjan@ubuntu:~/Devops/P3$ sudo dockerimages
sudo: dockerimages: command not found
niranjan@ubuntu:~/Devops/P3$ sudo docker images
REPOSITORY    TAG        IMAGE ID    CREATED    SIZE
p3            latest    936fb91f58dd    4 minutes ago    1.13GB
secfile       latest    227209cd2cf2    11 days ago    17.8MB
jenkins/jenkins    lts       8e6f48b11f82    3 weeks ago    498MB
niranjan@ubuntu:~/Devops/P3$ sudo docker rmi p3
Untagged: p3:latest
Deleted: sha256:936fb91f58dd5767855fb337301ec08a2497b1b9053130fe312a1b4054314ac4
niranjan@ubuntu:~/Devops/P3$ sudo docker images
REPOSITORY    TAG        IMAGE ID    CREATED    SIZE
secfile       latest    227209cd2cf2    11 days ago    17.8MB
jenkins/jenkins    lts       8e6f48b11f82    3 weeks ago    498MB
niranjan@ubuntu:~/Devops/P3$
```